

# RouteBridge Logistics: the full project guide

Everything you need to explain, defend and extend this project. Facts here come from the code in this repository (as of 8 October 2026).

## 1. The 60-second pitch

RouteBridge Logistics is a **last-mile delivery platform built for Nigeria**. A delivery company signs up, adds the shops it delivers for, and its riders. Orders flow in from shops (a dashboard, a CSV file, or an online-store webhook), get assigned to the nearest free rider (automatically or by a dispatcher), and the rider completes them from a phone app that **keeps working with no internet**. Customers follow their parcel on a live map. Cash on delivery is recorded and reconciled, shops get payout statements, and the company owner sees everything from one console.

**One sentence:** it replaces WhatsApp messages, phone calls and paper notebooks in delivery with one system that shows where every parcel and every naira is.

## 2. The problem it solves

| Problem in Nigerian last-mile delivery                                                       | How it hurts                                              | What RouteBridge does                                                                                                                                          |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Addresses are unreliable.</b><br>Few postcodes, landmarks like "opposite the yellow gate" | Riders get lost, deliveries fail, time and fuel wasted    | Address + landmark + Plus Code + map pin; each address gets a confidence score; customers and riders can correct the location (the original is kept)           |
| <b>Cash on delivery is dominant</b>                                                          | Cash gets lost or skimmed; nobody can prove who owes whom | COD amount per order, the rider records cash collected, mismatches open a <b>reconciliation</b> item, bank/gateway statements can be imported and matched      |
| <b>Riders have patchy data</b>                                                               | Apps that need internet fail in the field                 | The driver app is an <b>offline-first PWA</b> : actions are saved on the phone and synced later, with duplicate protection                                     |
| <b>"Where is my parcel?" calls</b>                                                           | Dispatchers spend the day answering phones                | Customer tracking page with live rider position, ETA, rider photo, and an alert when the rider arrives                                                         |
| <b>Shops have no visibility</b>                                                              | Disputes about money and damaged goods                    | Merchant portal: own orders, <b>payout statements</b> (cash collected minus fees plus approved claims), <b>claims</b> workflow (damage, loss, refund, dispute) |
| <b>No proof of delivery</b>                                                                  | "I never received it" disputes                            | Delivery code (OTP) given to the customer, rider's photo, recipient name, GPS, all timestamped in an                                                           |

| Problem in Nigerian last-mile delivery                 | How it hurts                        | What RouteBridge does                                                                          |
|--------------------------------------------------------|-------------------------------------|------------------------------------------------------------------------------------------------|
|                                                        |                                     | append-only audit trail                                                                        |
| <b>Dispatch by phone and WhatsApp</b>                  | Slow, error-prone, nothing recorded | Dispatch board, auto-assignment by distance/availability, live map                             |
| <b>Many small courier companies, one software need</b> | Each can't afford its own system    | <b>Multi-tenant SaaS:</b> every company has its own isolated workspace, with plans and billing |

### 3. What the product does (feature map)

**For the delivery company (the console)** - Overview with live numbers; Dispatch board; Orders; Drivers; Live map; Reconciliation; Settings. - Create orders by hand, import a CSV, or receive them from Shopify/WooCommerce-style webhooks (signed). - Assign a rider manually or switch on **automatic nearest-driver assignment** (never picks a busy or offline rider). - Settings: team members and roles, merchants, delivery zones and rate cards, dispatch behaviour, **plan and billing**, audit history. - Reports: summary, exceptions, per-rider payout CSV, merchant statements; bank-statement import for reconciliation. - Alerts: bell panel and a card when a rider is waiting for a delivery code.

**For riders (the driver app, /driver )** - Opens from a link or QR code from dispatch (no app store). Works offline; updates queue and sync. - Job list and detail, Navigate (Google Maps) and Waze buttons, **Call customer** (only after arriving), request a **delivery code**, complete delivery with code, photo and cash, record a failed attempt with a reason, correct the location, share GPS position. Optional phone alerts when assigned.

**For customers ( /track/<code> )** - Status timeline, live map with the rider and ETA, rider's first name and photo, "alert me when my rider arrives", add landmark/notes, opt out of texts. No sign-in needed (the link is the key; it expires).

**For shops (merchant portal)** - Own orders and numbers, create orders, **phone number required before creating an order**, payout statements with CSV, claims ("Problems"), optional email for alerts.

**For the platform owner (platform console)** - All companies, people, platform admins, system health, and a platform-wide audit trail.

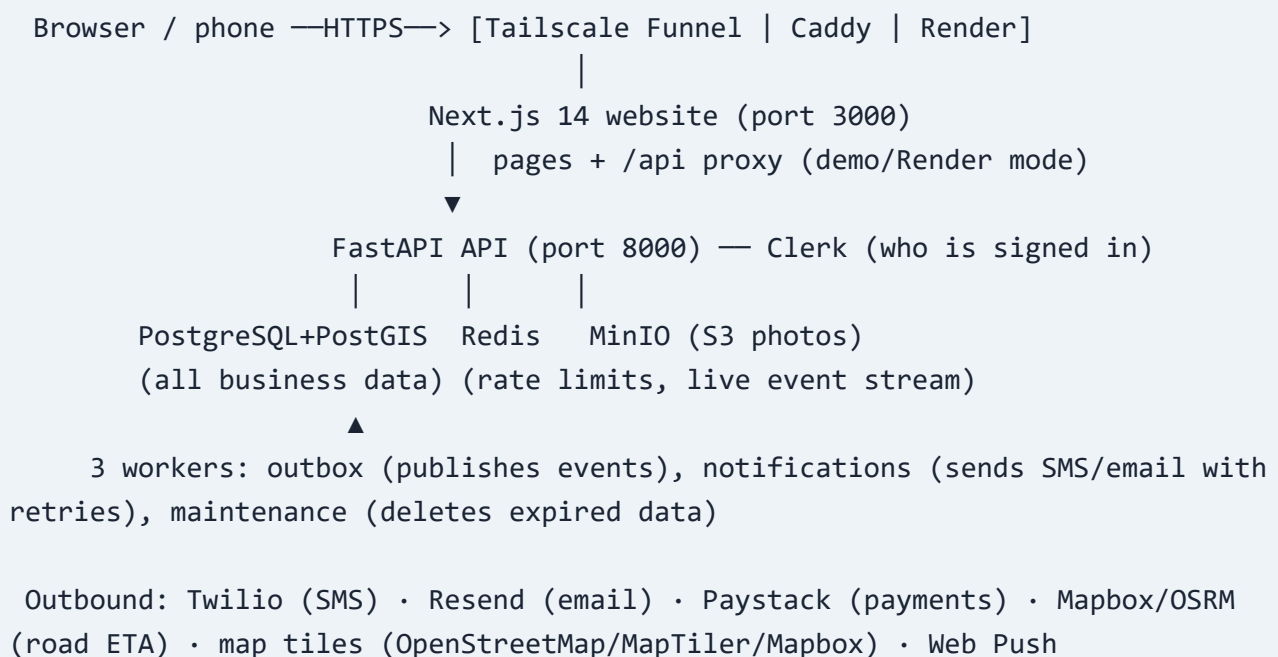
**Behind the scenes** - Notifications over SMS (Twilio), email (Resend), WhatsApp (adapter), and web push to riders; billing with Paystack; live updates over server-sent events; monitoring with Prometheus/Grafana; backups.

### 4. How it works: one delivery, start to finish

1. **Setup.** The platform admin signs in and the company owner gets a workspace. The owner adds **merchants, zones and rate cards**, and **drivers**; each driver gets an access link and QR code.

2. **An order arrives.** A shop creates it in its dashboard (its phone number must be saved first), or staff create it, or a CSV import / webhook does. The server creates the customer, the order, a delivery job, a stop (address and pin), a plan row (zone, time window) and a **tracking token**. The customer is texted a tracking link; the shop's phone/email gets an alert.
3. **Assignment.** Automatic (nearest available rider within a radius, never busy/offline) or by a dispatcher. The rider's phone gets a push alert; the outbox publishes events to the live stream.
4. **On the road.** The driver app moves the job `assigned` → `accepted` → `en_route` → `arrived`, sending GPS pings. The server keeps the latest position on the driver. Customers see the rider only while the parcel is en route/arrived and the ping is fresh (under 15 minutes). The ETA uses a road-time service if configured (Mapbox with traffic, or OSRM), otherwise a distance-based estimate.
5. **Arrival.** The customer is texted "your driver has arrived". The **Call customer** button becomes active (and only then): it opens the phone's dialer.
6. **Delivery code.** The rider taps "Text the customer a delivery code". With the free SMS setup, a card pops up on the dashboard: staff send the code to the customer (WhatsApp, text, call) and press "Mark as sent". The customer gives the code to the rider.
7. **Proof of delivery.** The rider enters the code, receiver's name, photo and cash collected. The server verifies the code (hashed, attempt-limited, expiring), records proof and payment, moves the job to `delivered`, and texts the customer.
8. **Money.** Cash collected is compared with what was due; differences become reconciliation items. Statements per shop and payout CSV per rider are generated from delivered jobs and rate cards.
9. **Problems.** A failed attempt records a reason and reschedules; shops raise claims; staff investigate, approve or reject; approved claims add to the shop's statement and the shop is notified.
10. **Billing.** The workspace is on a plan (trial by default). When limits are enforced, creating new riders/people/shops/orders beyond the plan is blocked with a clear message; paying through Paystack extends the plan.

## 5. Architecture



Ops: Docker Compose • Prometheus + Alertmanager + Grafana • Caddy (HTTPS) • backups

**Repository layout** - `backend/apps/api/src/routebridge/`: `routes/` (HTTP, 22 files), `services/` (business logic, 20), `models/` (database tables, 14 files, 36 tables), `auth/` (Clerk + roles), `workers/`, `integrations/` (Clerk, Svix, Paystack, circuit breaker, rate limit), `tools/` (migrate, checks, key generators), `config/` (settings, plans, production checks). Migrations: `backend/apps/api/alembic/versions/` (14). - `frontend/routebridge/apps/web/`: `app/` (pages), `components/` (19 screens), `lib/` (API client, offline queue, helpers), `public/` (service worker, manifest, images). - `backend/tests/`: 35 test files. `deploy/`: Caddy, monitoring, setup and backup scripts. `docs/`: guides. Root: `docker-compose.yml`, `render.yaml`, `.env.example`.

## 6. Technology used and why

### Backend

| Technology                              | Role                                            | Why it was chosen                                                                                                      |
|-----------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Python 3.12 + FastAPI</b>            | The API (about 114 endpoints)                   | Fast to build, automatic input validation and API docs, async support for live streams, great typing                   |
| <b>SQLModel</b> (Pydantic + SQLAlchemy) | Database models and request shapes in one place | One class describes the table and its validation; less duplication                                                     |
| <b>PostgreSQL + PostGIS</b>             | Main database                                   | Reliable, relational (money and orders need transactions), PostGIS gives real "nearest driver within X metres" queries |
| <b>Alembic</b>                          | Database migrations                             | Versioned, repeatable schema changes (14 so far), safe upgrades of live data                                           |
| <b>Redis</b>                            | Rate limiting, live event stream                | Fast shared counters and streams; powers sign-in throttling and the dashboard's live updates                           |
| <b>MinIO (S3 API)</b>                   | Delivery photos and signatures                  | Self-hosted S3-compatible storage; presigned URLs let phones upload straight to storage                                |
| <b>Clerk (+ PyJWT)</b>                  | Sign-in and identity                            | Secure auth without building passwords, 2FA or sessions; the API only verifies Clerk's signed tokens                   |
| <b>httpx</b>                            | Calls to Twilio, Resend, Paystack, routing      | Modern HTTP client, easy to fake in tests                                                                              |
| <b>pydantic-settings</b>                | Configuration from environment                  | Typed, validated settings; the production check refuses unsafe configs                                                 |
| <b>pywebpush</b>                        | Web push alerts to riders                       | Standard VAPID push without a native app                                                                               |
| <b>python-multipart, uvicorn</b>        | File upload parsing, the server                 | Required by FastAPI for forms/uploads; production ASGI server                                                          |

| Technology    | Role                | Why it was chosen                                                    |
|---------------|---------------------|----------------------------------------------------------------------|
| <b>pytest</b> | Tests (182 passing) | Simple, powerful; SQLite for speed, Postgres/PostGIS checks separate |

## Frontend

| Technology                                            | Role                           | Why                                                                                                                 |
|-------------------------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Next.js 14 (App Router) + React 18</b>             | The website and driver PWA     | One codebase for console, portal, tracking page and driver app; server rendering, routing, standalone Docker output |
| <b>TypeScript 5</b>                                   | All code                       | Catches mistakes before they reach users; shared types for API data                                                 |
| <b>@clerk/nextjs</b>                                  | Sign-in screens and session    | Matches the backend's Clerk; middleware protects pages                                                              |
| <b>Leaflet</b>                                        | Maps (live map, tracking page) | Light, free, works with OpenStreetMap, MapTiler or Mapbox tiles                                                     |
| <b>qrcode</b>                                         | QR code for driver access      | Riders sign in by scanning, no typing long tokens                                                                   |
| <b>Vitest + fake-indexeddb</b>                        | Unit tests (43)                | Fast tests; fake IndexedDB lets the offline queue be tested without a browser                                       |
| <b>ESLint (next)</b>                                  | Code quality                   | Catches common bugs and style problems                                                                              |
| <b>Service worker + IndexedDB</b> (browser built-ins) | Offline driver app             | Caches the app shell; stores queued actions on the phone                                                            |

## Platform and operations

| Tool                                            | Why                                                                                                               |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Docker Compose</b>                           | The whole system (database, cache, storage, API, 3 workers, website) starts with one command, the same everywhere |
| <b>Caddy</b>                                    | Automatic free HTTPS certificates and reverse proxy                                                               |
| <b>Prometheus + Alertmanager + Grafana</b>      | Metrics, alert rules (API down, errors, stuck events, failing messages), dashboard                                |
| <b>Tailscale Funnel / Cloudflare tunnel</b>     | Put the app on the internet from a home computer with no open ports (used for the live demo)                      |
| <b>render.yaml (Render Blueprint)</b>           | Ready-made cloud deployment on Render's free plan                                                                 |
| <b>Twilio / Resend / Paystack / Mapbox-OSRM</b> | SMS, email, naira payments, road-based ETA (each behind a switch; off until keys are added)                       |

## 7. Dependencies (exact, from the project files)

**Frontend (package.json)** - Runtime: `next ^14.2.5`, `react ^18.3.1`, `react-dom ^18.3.1`, `@clerk/nextjs ^6.25.0`, `leaflet ^1.9.4`, `qrcode ^1.5.4`. - Development: `typescript ^5.5.4`, `eslint ^8.57.0`, `eslint-config-next ^14.2.5`, `vitest ^2.1.9`, `fake-indexeddb ^6.2.5`, and type packages `@types/node`, `@types/react`, `@types/react-dom`, `@types/leaflet`, `@types/qrcode`. - Scripts: `dev`, `build`, `start`, `lint`, `test` (vitest).

**Backend (pyproject.toml)** - `fastapi >=0.115`, `uvicorn[standard] >=0.30`, `sqlmodel >=0.0.22`, `pydantic-settings >=2.6`, `psycopg[binary] >=3.2`, `alembic >=1.14`, `python-multipart >=0.0.12`, `PyJWT[crypto] >=2.9`, `httpx >=0.27`, `redis >=5.0`, `pywebpush >=2.0`. - Test extra: `pytest >=8.3`, `httpx`.

**Infrastructure images (compose):** `postgis/postgis:16-3.4`, `redis:7-alpine`, `pgsty/minio` (S3 storage), `caddy:2`, `prom/prometheus`, `prom/alertmanager`, `grafana/grafana`, `cloudflare/cloudflared` (demo tunnel).

## 8. Data model (36 tables, grouped)

- **People and access:** `user`, `tenantmembership` (role per workspace, optional merchant link), `platformadmin`, `tenant`, `tenantarea` (which operating areas a company serves).
- **Catalog:** `merchant`, `customer`, `servicezone`, `ratecard`, `operatingarea`, `country`.
- **Orders and delivery:** `order`, `deliveryjob`, `stop`, `jobplan`, `driver`, `driverassignment`, `stopcorrection`, `trackingtoken`, `deliveryotp`, `proofofdelivery`, `deliveryattempt`.
- **Money:** `paymentrecord`, `reconciliationitem`, `billingpayment`, `claim`.
- **Messaging:** `notification`, `notificationdelivery`, `consentrecord`, `pushsubscription`.
- **Reliability and audit:** `outboxevent`, `auditevent` (append-only), `platformauditevent` (what platform staff did), `webhookreceipt`, `idempotencyrecord`, `mobilesyncnevent`.

## 9. Engineering decisions worth knowing

- **Multi-tenancy:** every business row carries `tenant_id`; every route checks the signed-in person belongs to that tenant and has an allowed role. Merchants are fenced to a portal scoped to their one shop.
- **Roles:** `tenant_owner`, `tenant_admin`, `dispatcher`, `operations_manager`, `finance`, `merchant_user`, `partner_operator`, `read_only`, `driver`; plus platform admins.
- **Outbox pattern:** events are saved in the same database transaction as the change, then a worker publishes them. Nothing is lost if Redis is down.
- **Idempotency:** creating orders accepts an `Idempotency-Key`; webhooks are de-duplicated by event id; the driver's offline actions carry unique ids so retries cannot double-apply.
- **Circuit breaker + retries:** outbound calls (SMS, email, routing) stop hammering a service that is down; failed messages retry with backoff.

- **Signed webhooks:** Clerk (Svix), Paystack (HMAC-SHA512), and order intake (HMAC with timestamp) are verified before they are trusted.
- **Privacy:** customers' phone numbers are masked in the driver app; delivery codes are hashed; message bodies are wiped after sending; photos and records expire through the maintenance worker.
- **Production safety:** with `ENVIRONMENT=production` the API refuses to start on unsafe settings (missing secrets, localhost origins, test email sender...), listing what is wrong.
- **Offline-first driver app:** an encrypted local store, an action queue in IndexedDB, background sync with exponential backoff, and flags for rejected updates.
- **Free-first design:** every paid vendor is optional behind a switch (SMS, email, traffic ETA, payments, maps); the app runs with none of them.

## 10. Testing and quality

- **182 backend tests** (pytest) and **43 frontend tests** (vitest); type-check ( `tsc` ) and lint must be clean.
- Real-browser checks (Playwright) were used to look at every screen, including phone-size layouts, using pretend data where real data should not be touched.
- Database checks: backup-then-restore comparison, migrations from an empty database, PostGIS queries versus the plain calculation.

## 11. What is not finished (be honest about this)

- Text messages: the Twilio account is a trial with no sender, so SMS to customers is not live; delivery codes use the dashboard relay instead.
- Production sign-in needs a domain you own; the live demo uses Clerk's development instance.
- The driver app is a PWA, not a native Android app.
- Real money (Paystack), phone push and road-time ETA have been tested with fakes, not with live accounts.
- Plan prices are placeholders; privacy documents are drafts needing legal review; capacity under load has not been measured.

## 12. Interview questions (with short model answers)

**About the product**

1. *What problem does RouteBridge solve and for whom?* Delivery companies, shops, riders and customers in Nigeria: unreliable addresses, cash on delivery, poor connectivity, and no visibility. One system replaces calls and notebooks.
2. *Who are the user types and what does each see?* Platform admin, company staff (owner/admin/dispatcher/ops/finance), merchants, riders, customers: each has a different screen and permissions.
3. *Why cash-on-delivery reconciliation?* COD is the main payment method; without matching cash collected to what was due, money leaks and disputes follow.
4. *How would you make money from it?* Subscription plans (trial, starter, growth, business, enterprise) with limits, paid via Paystack in naira.

**Architecture** 5. *Draw the architecture.* Browser/phone → Next.js → FastAPI → PostgreSQL/PostGIS, Redis, MinIO; workers; Clerk; outside services. 6. *Why FastAPI and Next.js?* FastAPI: typed, validated, fast to build and document; Next.js: one app for several audiences with routing and server rendering. 7. *Why PostgreSQL with PostGIS?* Money needs transactions; PostGIS answers "nearest rider" efficiently (with a plain fallback). 8. *What is the outbox pattern and why use it?* Save the event with the data in one transaction, publish later; avoids losing or duplicating events when a broker is down. 9. *How do live updates reach the dashboard?* Server-sent events from a Redis stream; the page refreshes its data on each event. 10. *Why separate worker processes?* Sending messages and publishing events can be slow or fail; keeping them out of web requests keeps the API fast, and they retry independently.

**Backend and data** 11. *How do you keep one company from seeing another's data?* `tenant_id` on every row, a membership check on every route, merchant users restricted to a portal scoped to one merchant, and tests that try cross-tenant access. 12. *How do you handle retries without duplicates?* Idempotency keys for orders, event ids for offline sync, unique references for payments, webhook de-duplication. 13. *How are database changes deployed safely?* Alembic migrations, a locked migration runner at start, backup before migrating, and a test that migrations run from an empty database. 14. *How do you verify a payment?* Never trust the browser: ask Paystack, check the amount matches the plan price, apply once per reference; the webhook is signature-checked. 15. *What is a circuit breaker?* It stops calling a failing service for a while so failures don't pile up; calls resume after a cool-down.

**Security** 16. *How is authentication done?* Clerk issues signed tokens; the API verifies them with the provider's public keys; riders use short-lived signed device tokens. 17. *How do you protect delivery codes?* Hashed with a per-job key, expire, attempt-limited, wiped on use; in relay mode the plain copy is kept only until used/expired and only staff can read it. 18. *What do you do about secrets?* Environment files that are never committed, production checks that refuse placeholders, a secret scan before each commit, and rotating anything that was shared. 19. *How do you prevent abuse of public pages?* Unguessable expiring tracking tokens, rate limits, escaped output, and a limited slice of data shown to customers. 20. *What would you do about spreadsheet injection in CSV exports?* Neutralise values that start with `=`, `+`, `-` or `@` (done in the statement export).

**Frontend and offline** 21. *How does the driver app work offline?* A service worker caches the shell; actions are stored in IndexedDB with unique ids; a sync loop uploads them with backoff; the server de-duplicates. 22. *How do you handle a rider's phone being lost?* Tokens are short-lived and revocable; local data is encrypted; dispatch can sign the device out. 23. *Why did you use a built-in proxy in demo mode?* So one public address serves both the website and the API (no CORS, one tunnel). 24. *How do you test UI that depends on location or notifications?* Fake the API replies in the browser, use fake IndexedDB in unit tests, and keep logic in small pure functions.

**Operations and decisions** 25. *How do you deploy it?* Docker Compose with Caddy for HTTPS on a server; or Render with the provided blueprint; or a tunnel from a computer for demos. 26. *How do you know it is healthy?* Health/ready endpoints, Prometheus metrics, alert rules, Grafana dashboard, and a backup-restore test. 27. *What would break first at scale and how would you fix it?* A single API process and database connections: run several API instances, add connection pooling, move workers out of the API, index hot queries, load-test first. 28. *Tell me about a bug you found and fixed.* Example: the sign-in guard did not wait for Clerk's promise, so signed-out visitors saw an empty page; fixed by awaiting it and adding a regression check. 29. *What trade-offs did you make because of cost?* Optional vendors,

dashboard relay for delivery codes instead of SMS, a tunnel instead of a server, Clerk development keys instead of a production domain. 30. *What would you build next?* A native Android app, live SMS and payments, load testing, a security review, and local-language support.